演習02：敵の探索・追跡AI

担当：c0a25107a3 / ブランチ C0A25107/enemy-ai

到達目標

敵が「見えた相手を追う」「見失った場所を探す」「見つからなければ巡回へ戻る」を使い分けるようにします。足音を聞いた場合は、音がした場所を調査します。

既存Guardにある巡回、単純追跡、幅優先探索は共通ベースです。新規部分は明示的な捜索状態、捜索地点の選択、足音への反応です。

先に00_commonの4〜8節を読みます。現在のGuard.updateのplayer引数はVector2ですが、新しいAPIはPlayerオブジェクトです。player.position と player.hidden を使います。

練習1：状態と記憶を決める

enemy_ai.pyを作り、既存Guardを移します。色などの定数はsettings.py、共通型はcontracts.pyから読みます。game.pyをimportすると循環するので避けます。

```
from enum import Enum, auto

class EnemyState(Enum):
    PATROL = auto()
    CHASE = auto()
    SEARCH = auto()
```

必要な状態はstate、last_seen、search_center、search_remaining、search_targets、next_cellです。route_index、position、stunnedは既存を使います。初期値はstate=PATROL、last_seen=None、search_center=None、search_remaining=0.0、search_targets=[]、next_cell=Noneです。再挑戦時はGuardを新しく作って初期化します。

search_remainingは捜索地点へ到着してから3秒を目安に減らします。移動中にも減らす設計にする場合は、遠い音に向かう途中で巡回へ戻らないよう相談してください。

練習2：状態遷移を書く

現在	条件	次の状態・記憶
どの状態でも	主人公が見える	CHASE、last_seenを更新
CHASE	見えなくなった	SEARCH、最後の目撃地点を固定
PATROL	聞こえる足音がある	SEARCH、音の位置を固定
SEARCH	新しい足音だけ聞こえる	必要なら音の位置へ捜索先を更新
SEARCH	到着後の捜索時間が終了	PATROL、通常巡回へ戻る

優先順位は「stunnedで停止」「視認」「足音」「それ以外」です。見えている主人公を追う最中に、古い足音へ戻らないようにします。

状態を変える処理を enter_search(cell) のようなメソッドへまとめると、タイマー初期化を毎フレーム行う誤りを避けられます。

練習3：視認を判定する

```
def can_see(self, player, world) -> bool:
    if player.hidden:
        return False
    near = self.position.distance_to(player.position) < 185
    return near and world.visible(self.position, player.position)
```

これは条件式の例です。185ピクセルは仮の値なので定数へ移し、全員で調整します。

見えている間だけlast_seenを更新します。見失った後もplayer.positionを目的地へ代入し続けると、敵が壁越しに現在地を知ってしまいます。

確認：壁の角で視線を切ると、その後に主人公が動いてもsearch_centerは最後の目撃地点のまま。

練習4：足音を受け取る

updateの形を update(dt, player, world, noises) に揃えます。noisesはそのフレームに発生したNoiseEventのtupleです。

各音のpositionをVector2に変換し、敵との距離がradius以下なら候補にします。複数聞こえるときは最も近い音を1つ選び、そのマスをsearch_centerにします。

```
for noise in noises:
    heard_position = pg.Vector2(noise.position)
    distance = self.position.distance_to(heard_position)
    # TODO: 半径内の候補から最も近い音を選ぶ
```

今回の提案では音は壁を通して届きます。視線とは別ルールです。全員で変更する場合は主人公資料にも反映します。隠れている主人公から新しい足音は出ません。

足音が新しい目標になっても、移動中のnext_cellへ到着してから経路を選び直します。斜めに壁の角を横切らないためです。

練習5：最後に見た場所を探す

まずworld.find_pathでsearch_centerへ移動します。到着したら、その上下左右から通れるマスを候補にして順に向かいます。敵の位置から経路が得られる候補だけを残します。

初回は候補を固定順にして、再現しやすくします。候補がない場合はその場で捜索時間を進め、時間切れでPATROLへ戻します。毎フレーム新しい候補を作り直さないでください。

start==goalなら到着扱いです。異なるのに経路が空なら到達不能です。捜索中心へ到達不能ならその捜索を終了してPATROLへ戻します。周辺候補へ到達不能ならその候補を捨てます。巡回目的地にも届かない場合は移動を止め、0.5秒後に再探索する設計にします。Noneへ向かって移動したり、同じフレーム内で無限に探索し直したりしないようにします。

PATROLへ戻るとき、元のroute_indexの目的地へ歩きます。近い巡回地点から再開する実装は発展課題です。

練習6：移動と接触を接続する

既存Guardの「次のマス中心へ移動する」部分を再利用します。移動速度はPATROL 88、CHASE 132、SEARCH 100ピクセル/秒などから試します。

HPはGame.update_guards側でPlayer.take_damageを呼びます。敵AI側では減らしません。隠れている相手への接触ダメージもGame側が止めます。

Game側の guard.update(dt, self.player.position) を次の形に置き換えます。

```
guard.update(dt, self.player, self.world, noises)
```

描画は巡回が紫、追跡が赤、捜索が黄などで区別します。状態名をデバッグ表示してから、最終的なアイコンに整えると確認しやすくなります。

練習7：相手の機能が未完成でも試す

starter/sandbox.pyをゲームのルートへ練習用にコピーすると、SandboxWorldとSandboxPlayerで判定を試せます。練習ファイルは完成後のゲームへ混ぜないでください。

```
world = SandboxWorld()
player = SandboxPlayer(world.center((3, 1)))
guard = Guard([(1, 1), (1, 3)], world)
guard.update(0.1, player, world, ())
assert guard.state is EnemyState.CHASE
```

この例はあなたの実装後に成立する確認です。まだGuardの引数やstateを変更していない段階では動きません。

次にplayer.hidden=Trueへ変え、SEARCHに移ることを確認します。主人公を遠くへ移し、音だけ与えるケースも別に試します。

動作確認表

準備・操作	期待する結果
近距離で壁なし、hidden=False	CHASEになる
同じ距離でも壁あり	新しく視認しない
追跡中に隠れる	最終目撃地点のSEARCHへ移る
見失った後に主人公を動かす	search_centerが追従しない
半径内の足音のみ与える	音の地点を捜索する
半径外の足音のみ与える	巡回を続ける
捜索中に再発見	CHASEへ戻る
候補なし・時間切れ	停止せずPATROLへ戻る
Escで停止・再開	捜索時間が停止中に減らない

よくある不具合と提出

敵が壁を抜ける場合は、目的地を変えるたびに座標を直接移動していないか、next_cellの中心へ着いてから曲がっているかを確認します。

敵がいつまでも捜索する場合は、enter_searchを毎フレーム呼んでタイマーを戻していないかを確認します。敵が瞬間移動する場合は「目的地」と「現在位置」を同じ変数にしていないかを見ます。

enemy_ai.pyとREADMEの担当欄をコミットします。PRには状態遷移、足音の仕様、接続API、確認表の結果を記載します。

基準コード：https://github.com/kusupon/ProjExD_Group13/blob/33fe9905c183b321d1988d66041feb8b7e74998f/game.py